

Restatify Booking API

Docker Compose mit VPN + HTTPS (Ubuntu 24.04)

Stand: 2026-04-05

Ziel: Produktionsnaher Betrieb mit Docker Compose, Host-Level WireGuard und TLS via Caddy.

1. Zielarchitektur

- Server A: WordPress
- Server B: Booking API (Docker Compose)
- WireGuard Tunnel zwischen beiden Hosts
- Caddy terminiert HTTPS und leitet intern an API weiter

2. Voraussetzungen

- Ubuntu 24.04 auf beiden Servern
- Docker Engine + Compose Plugin auf API-Host
- DNS: booking-api.deine-domain.tld auf API-Host
- Offene Ports: 22/tcp, 80/tcp, 443/tcp, 51820/udp

2.1 Docker Engine + Docker Compose via apt installieren

Empfehlung: offizielles Docker Repository fuer Ubuntu 24.04 verwenden.

```
sudo apt remove -y docker.io docker-doc docker-compose docker-compose-v2 podman-docker
sudo apt update
sudo apt install -y ca-certificates curl gnupg
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/
sudo chmod a+r /etc/apt/keyrings/docker.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg]
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker
sudo systemctl enable --now docker
sudo docker version
sudo docker compose version
```

Optional fuer Betrieb ohne sudo:

```
sudo usermod -aG docker $USER
```

Danach neu einloggen, damit die Gruppenmitgliedschaft greift.

3. Docker Compose (Produktion)

```
services:
  db:
    image: postgres:16-alpine
    restart: unless-stopped
    environment:
      POSTGRES_DB: restatify_booking
      POSTGRES_USER: restatify_booking
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    volumes:
      - pgdata:/var/lib/postgresql/data

  api:
    build:
      context: .
      dockerfile: Dockerfile
    restart: unless-stopped
    env_file:
      - .env.local
    volumes:
      - booking_data:/app/data
    expose:
      - "8088"

  caddy:
    image: caddy:2-alpine
    restart: unless-stopped
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ./Caddyfile:/etc/caddy/Caddyfile:ro
      - caddy_data:/data
      - caddy_config:/config

volumes:
  pgdata:
  booking_data:
  caddy_data:
  caddy_config:
```

4. Caddy TLS Konfiguration

```
booking-api.deine-domain.tld {  
    encode zstd gzip  
    reverse_proxy api:8088  
}
```

Zertifikate werden automatisch von Let's Encrypt ausgestellt.

5. WireGuard (Host-Ebene)

```
sudo apt install -y wireguard  
sudo systemctl enable --now wg-quick@wg0  
sudo wg show
```

WireGuard laeuft auf den Hosts, nicht als Compose-Service.

6. HTTPS + VPN gemeinsam nutzen

Variante A (einfach)

- Plugin nutzt `https://booking-api.deine-domain.tld` (oeffentliche Route, TLS-gesichert).

Variante B (bevorzugt)

- Plugin nutzt weiterhin die HTTPS-Domain.
- Auf WP-Host Domain lokal auf VPN-IP mappen.

```
10.44.0.1 booking-api.deine-domain.tld
```

Ergebnis: TLS bleibt gueltig, Transport laeuft durch den WireGuard-Tunnel.

7. Deployment

1. .env.local und Caddyfile anpassen
2. Compose starten:

```
docker compose -f docker-compose.prod.yml up -d --build
```

3. Health pruefen:

```
curl https://booking-api.deine-domain.tld/health
```

4. WordPress Plugin konfigurieren (URL + API Key)

8. Betrieb

```
docker compose -f docker-compose.prod.yml logs -f api  
docker compose -f docker-compose.prod.yml logs -f caddy  
docker compose -f docker-compose.prod.yml ps
```

Secret-Rotation ohne Downtime bleibt als geplanter Security-Folgepunkt bestehen.